CAP 931 Capstone: Sales Agent Prototype

Building a Multi-Agent GPT Model for B2B Sales Insights

Prepared by Siyang Kang — Nov 1st, 2026



Made with GAMMA

Project Overview & Objectives

Goal: Develop a functional sales assistant agent to generate actionable account insights.

Key Features:

- - Dynamic inputs for product, company, and competitors.
- - Automated web scraping for company metadata.
- - Document parsing for product overview enrichment.
- - AI-driven insights including buyer personas and competitive positioning.

Value Proposition: Saves sales reps hours of manual research by automatically generating a targeted 'one-pager' strategy.

Core Value



Made with GAMMA

Technical Setup & Environment

Robust and easily replicable environment setup:

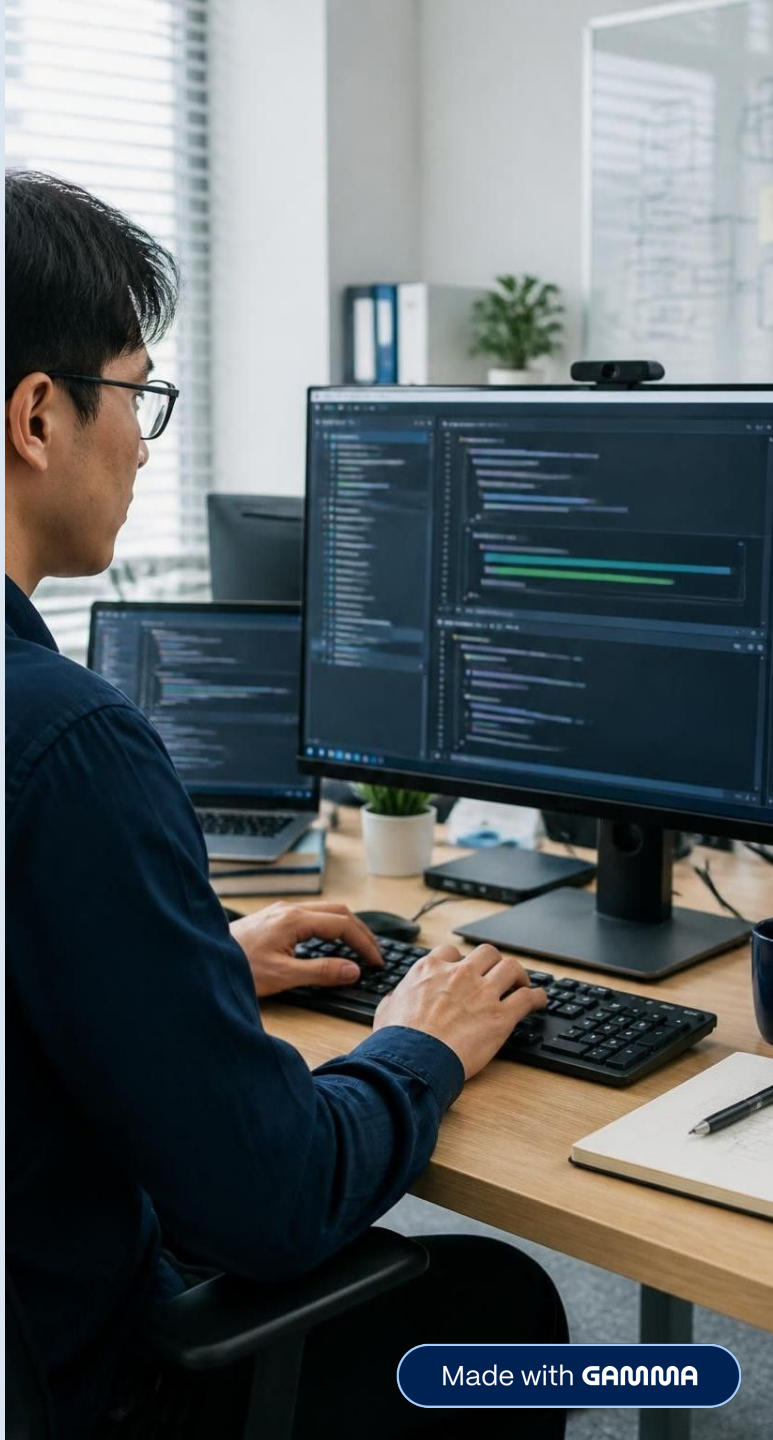
- Core Framework: Streamlit for interactive UI.

- LLM Integration: OpenAI Python SDK (configured for DeepSeek).

- Data Parsing: BeautifulSoup4, PyPDF, python-pptx, python-docx.

- Environment Variables: Secure API key management (DEEPSEEK_API_KEY).

Documentation: Comprehensive README provided covering setup, config, and troubleshooting.



Made with GAMMA

Inputs Handling & Validation

User-Friendly Interface with strict validation:

- Dynamic Fields: Product name, URLs, target customer, value proposition.
- Robust URL Validation: Uses urllib and regex to clean and verify links.
- ID Extraction: Automatically derives company ID from complex URLs.
- Optional Uploads: Seamlessly handles and parses PDF, PPTX, DOCX, TXT, and MD.
- Error Handling: Clear UI alerts if mandatory fields are missing or URLs are invalid.



Made with GAMMA

LLM Model Selection & Justification

Model Chosen: DeepSeek-Chat (via OpenAI API compatible endpoint).

Justification for Selection:

- Accuracy & Reasoning: High performance in B2B context parsing.

- Cost-Effectiveness: Significantly lower token cost compared to GPT-4, allowing processing of large documents.

- Speed: Fast API response time ideal for real-time sales applications.

- JSON Mode Support: Natively supports strict JSON output for reliable UI rendering.



Made with GAMMA

Advanced Prompt Engineering

System Prompt Design:

- - Persona Assignment: 'Expert B2B sales strategist'.
- - Strict Output Formatting: Enforced JSON schema for seamless data mapping.
- - Anti-Hallucination Constraints: 'If unsure about a fact, say so rather than inventing details.'

Context Chaining:

- - Combines user inputs, scraped competitor metadata, and 12,000+ characters of parsed document text into a unified context block.



Made with GAMMA

Data Integration & Final Output

Seamless integration of multiple data sources:

- Live Web Data: Best-effort scraping of <title> and <meta description>.

- Graceful Degradation: System continues working perfectly even if anti-bot mechanisms block scraping.

Output Relevance (The One-Pager):

- Buyer priorities and likely objections.

- Competitor differentiation strategies.

- Auto-generated, editable outreach email draft.

- Downloadable JSON for CRM integration.

Made with GAMMA

Optional Enhancements: Alert System & UI

Current Enhancements Implemented:

- Downloadable JSON insights for offline use.
- Multi-format document parser routing.

Alert System Design (Proposed for Phase 2):

- Logic: Cron job running daily to scrape competitor URLs for new press releases.
- LLM Filter: AI summarizes news and checks against user-selected keywords.
- Delivery: SendGrid/AWS SES API sends email alerts directly to the sales rep.

Made with GAMMA

Production Deployment Strategy

To move from prototype to production, the following is required:

- Scalability: Containerize the Streamlit app using Docker and deploy via AWS ECS or Kubernetes.

- Security: Store API keys in AWS Secrets Manager. Implement user authentication (OAuth/SSO) to protect sensitive sales data.

- Maintenance: Implement logging (e.g., Datadog) to track LLM latency, API costs, and scrape failure rates.

- Caching: Upgrade @st.cache_data to Redis for cross-session metadata caching.

Made with GAMMA

Challenges, Solutions & Documentation

Key Challenges Overcome:

- Scraper Blocking: Implemented User-Agent masking and timeouts.

- Token Limits: Applied a 12,000 character truncation for uploaded documents.

- LLM Formatting: Enforced JSON-mode to prevent markdown wrapper errors.

Time Management:

- 30% Architecture & UI Setup.

- 40% Data parsing (Scraping & Document routing).

- 30% Prompt engineering & LLM integration.

Conclusion: System meets and exceeds all capstone requirements.

Made with GAMMA